



Bohdan DYSHLEVYY
Etudiant de BUT Réseaux et
Télécommunications
IUT NICE CÔTE-D'AZUR
PROMOTION BUT2 TRAD Année
2024-2025

Configuration automatique du réseau expérimental SLICES-RI

au Centre Inria d'Université Côte d'Azur, Équipe-projet DIANA

Maître de stage :
Damien SAUCEZ
Chercheur INRIA
Centre Inria d'Université Côte
d'Azur

Tuteur pédagogique :
Guillaume URVOY-KELLER
Professeur des Universités
à l'Université Côte d'Azur
IUT NICE CÔTE D'AZUR



Plan de la présentation

1. Contexte de recherche : SLICES et FABRIC
2. Objectifs et rôle du stagiaire
3. Automatisation des tests sur FABRIC
4. **Génération automatique et déploiement de configuration VLAN**
5. Intégration : API, Ansible, Docker
6. Tests finaux et résultats
7. Conclusion et perspectives
8. Remerciements

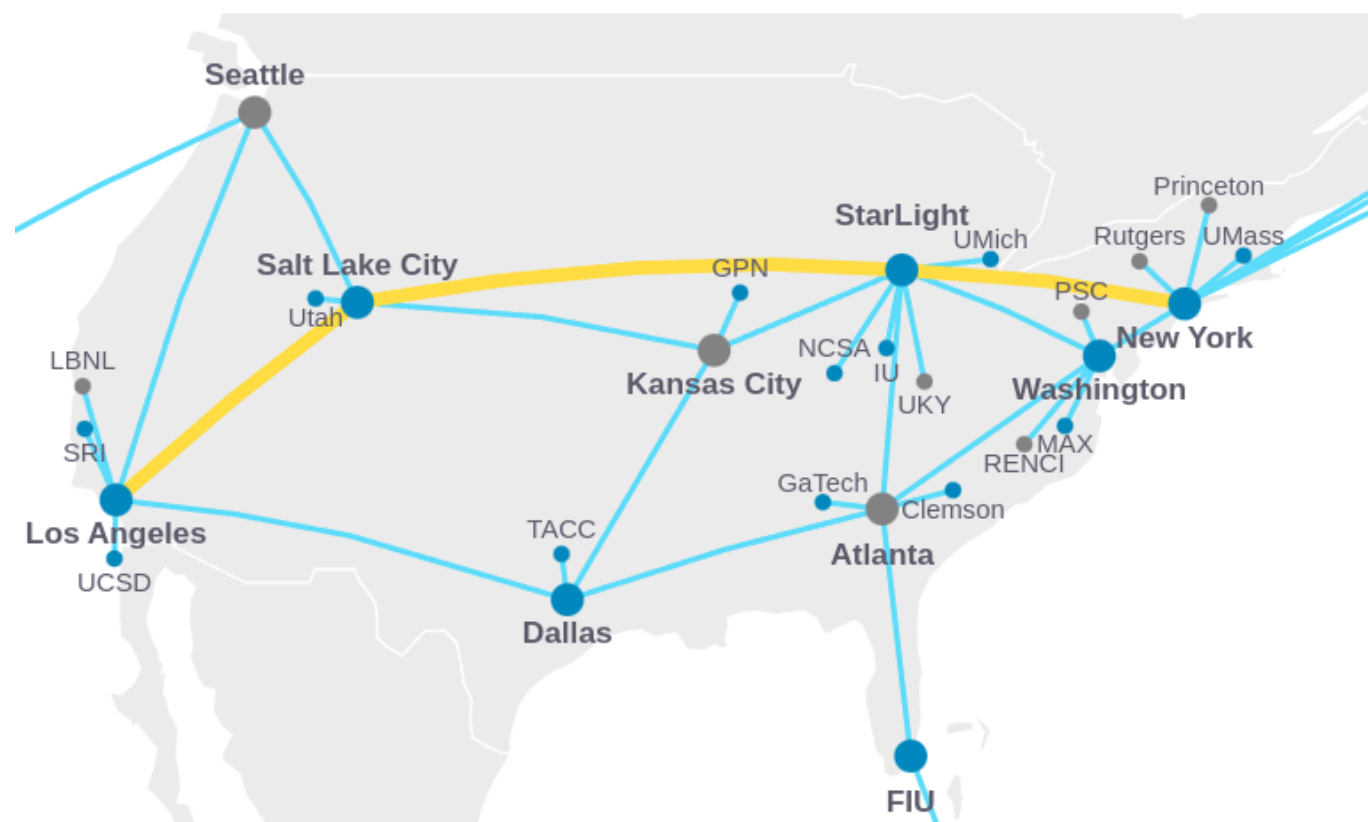
Contexte -SLICES-RI (EU)

- SLICES-RI : plateforme européenne de recherche pour tester des architectures réseaux (5G/6G, IoT, cloud, sécurité).
- Objectif : permettre des expériences répétables sur des ressources réparties en Europe et à l'international.
- Notion de slice : réservation de ressources réseau isolées.



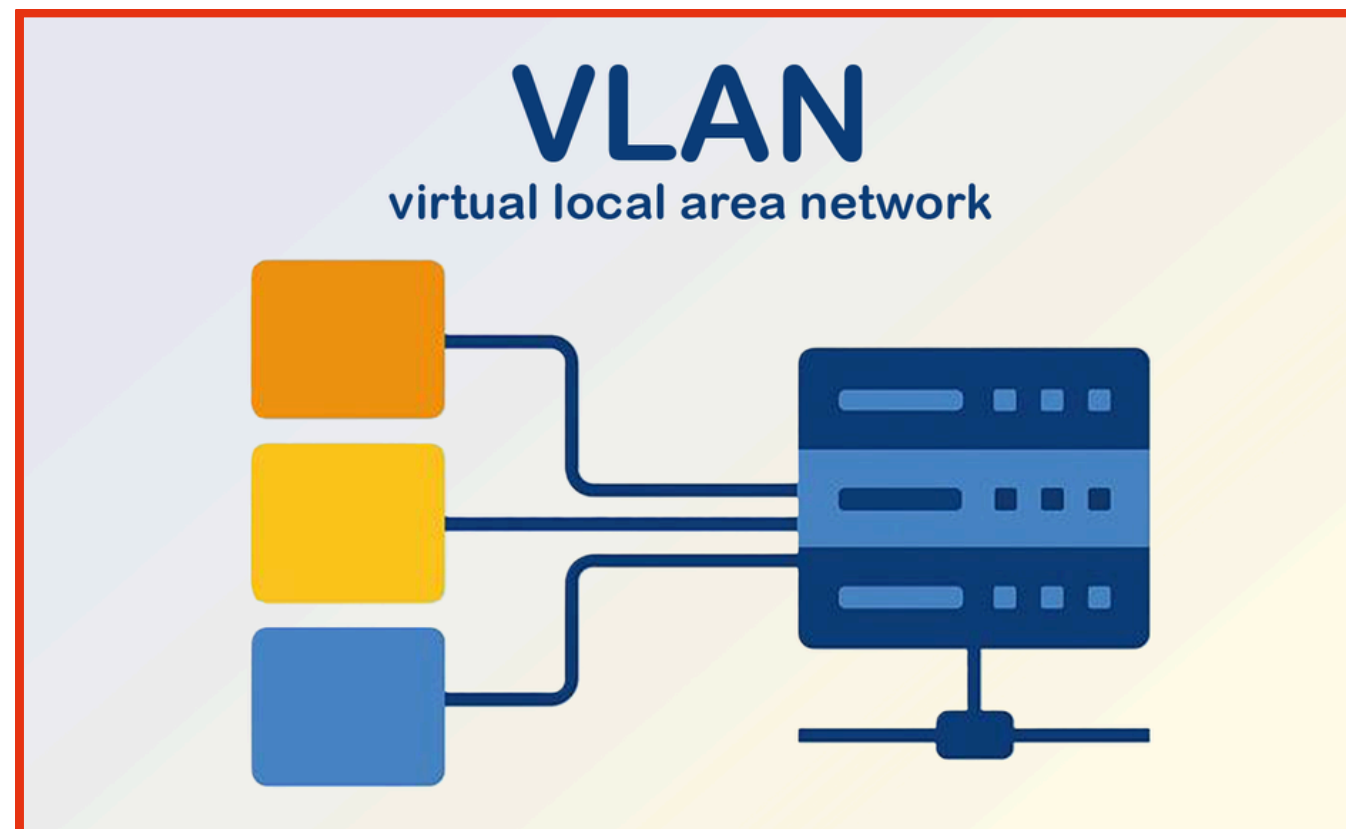
Contexte - FABRIC Testbed (USA)

- Plateforme américaine similaire à SLICES, mais se concentrant uniquement sur réseaux filaire optique.
- Basée sur Internet2, réseau très haut débit pour la recherche.
- Utilisée pour faire des tests de performance sur liens longues distances.



Problématique

- Dans SLICES : besoin d'isoler le trafic entre expériences.
- Nous avons décidé d'utiliser VLAN mais y a plusieurs de solutions.
- La configuration des équipements est souvent manuelle, lente et sujette à erreurs.



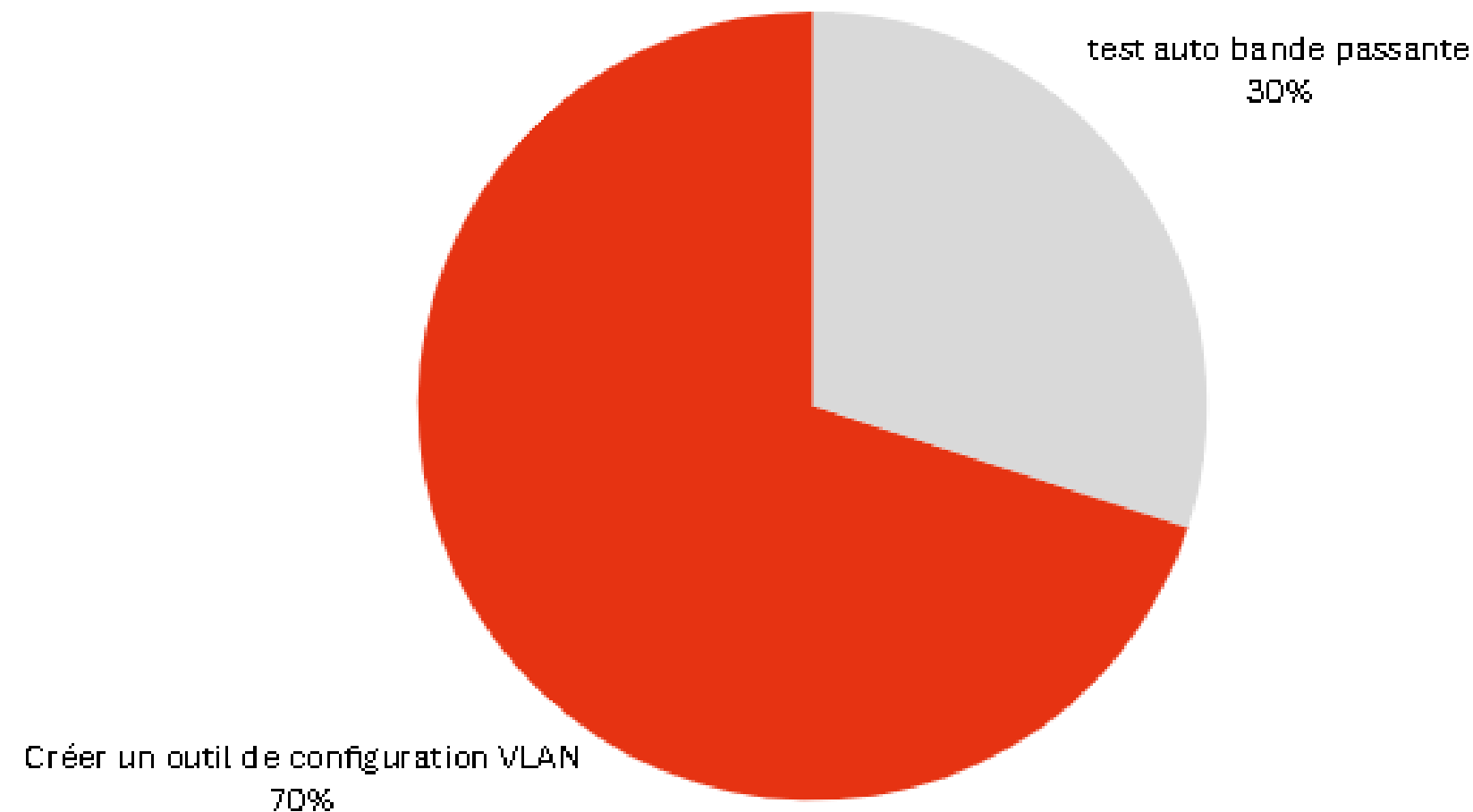
Objectif du stage : automatiser la génération et le déploiement de configurations VLAN, pour assurer l'isolation entre expériences.

Mon rôle de stagiaire

Intégration à l'équipe DIANA (Centre Inria d'Université Côte d'Azur)

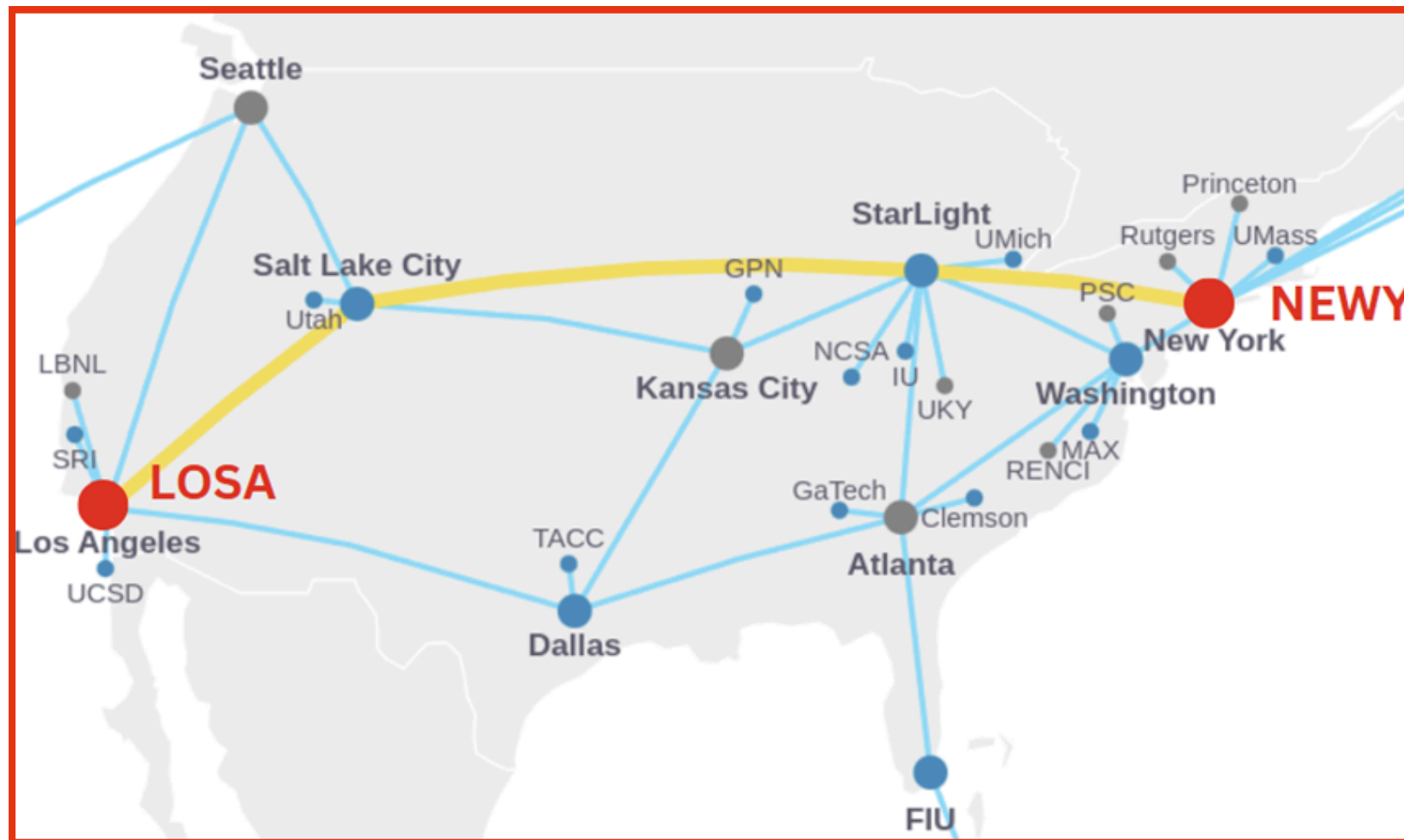
Missions principales :

- Automatiser le test bande passante sur FABRIC (30 % du temps, début avril - fin avril)
- Créer un outil qui génère et déploie les configuration VLAN (70 % du temps, fin avril - fin juin)



Automatisation des tests de bande passante sur FABRIC

- Script Python utilisant fabrictestbed-extensions.
- Test de bande passante entre deux sites de FABRIC (New York et Los Angeles).
- Utilisation de iperf, résultats enregistrés pour analyse (analyse non couverte par le stage).



```
1
2 [2025-04-14 14:02:57] Bandwidth test:
3
4 Client connecting to 10.20.30.41, TCP port 5001
5 TCP window size: 85.0 KByte (default)
6
7 [ 3] local 10.20.30.40 port 42312 connected with 10.20.30.41 port 5001
8 [ ID] Interval      Transfer    Bandwidth
9 [ 3]  0.0-10.0 sec   415 MBytes  348 Mbits/sec
10
```

Intégration dans GitLab CI/CD



- Création d'un pipeline automatisé avec scheduler pour lancer les tests.
- Utilisation de variables masquées pour la sécurité.
- Création d'un runner sur la plateforme CI d'Inria.
- Sauvegarde automatique des résultats dans des artéfacts GitLab.

CI/CD Variables </> 9 Reveal values Add variable

Key ↑	Value	Environments	Actions
FABRIC_BASTION_HOST	*****	All (default)	✎ 🗑
FABRIC_BASTION_KEY_BASE64	*****	All (default)	✎ 🗑
Protected Masked			
FABRIC_BASTION_USERNAME	*****	All (default)	✎ 🗑
FABRIC_CREDMGR_HOST	*****	All (default)	✎ 🗑
FABRIC_ORCHESTRATOR_HOST	*****	All (default)	✎ 🗑
FABRIC_PROJECT_ID	*****	All (default)	✎ 🗑
FABRIC_SLICE_KEY_BASE64	*****	All (default)	✎ 🗑
Protected Masked			
FABRIC_SLICE_PUBKEY_BASE64	*****	All (default)	✎ 🗑
Protected			
FABRIC_TOKEN_BASE64	*****	All (default)	✎ 🗑
Protected Masked			

slices

Public Active Slaves Members Logs

Quotas usage

CPU : 2 / 40 cores

Memory : 4096 / 40960 MB

Primary storage : 50 / 400 GiB

Secondary storage : 0 / 400 GiB

If you want to get more actions / information on slaves (add a disk, create a template, etc.), you can access CloudStack using the same credentials as the CI portal. The Domain must be ci/slices.

[CloudStack](#)

Status

Running

Hostname

slices-huge

IP

172.21.12.232

Description

slices-huge slave

Edit

CPU/Memory

2*2 Ghz / 4 Gb

Created

15/04/2025 16:28

Expire

30/04/2026 11:27

Extend

Actions

Connect

Stop

Delete

Add slave

Intégration dans GitLab CI/CD



```
509 -----
510 Client connecting to 10.20.30.41, TCP port 5001
511 TCP window size: 85.0 KByte (default)
512 -----
513 [ 3] local 10.20.30.40 port 47656 connected with 10.20.30.41 port 5001
514 [ ID] Interval      Transfer    Bandwidth
515 [ 3]  0.0-10.0 sec  399 MBytes  334 Mbits/sec
516 Bandwidth test complete. Results saved.
517 Uploading artifacts for successful job                                00:01
518 Uploading artifacts...
519 fabric_bandwidth_log.txt: found 1 matching artifact files and directories
520 Uploading artifacts as "archive" to coordinator... 201 Created id=5631526 responseStatus=201 Create
    d token=64_BZycvy
521 Cleaning up project directory and file based variables                00:00
522 Job succeeded
```

Artifacts

Total artifacts size 27.83 MiB

☐ Artifacts	Job	Size	Created	
☐ 3 files	bandwidth-test archive +2 more #1185110 62e65176 main	45.20 KiB	53 minutes ago	
☐ artifacts.zip	archive	439 B		
☐ metadata.gz	metadata	169 B		
☐ job.log	trace	44.60 KiB		

Générateur de configuration VLAN

- Représentation du réseau avec NetworkX (graphes).
- Fichier .dot comme entrée pour topologies physique et virtuelle.
- Chaque lien logique = 1 VLAN (sauf unmanaged switch).
- Interfaces configurées en trunk / access automatiquement selon le contexte.

topo virtuelle

```
graph G {
    "sopnode-f3" [type="host"];
    "sopnode-f1" [type="host"];
    sw1 [type="unmanaged"];

    "sopnode-f1" -- sw1;
    "sopnode-f3" -- sw1;
}
```

```
vlan 4000
 name VLAN4000
 !
interface Eth1/44
 description sopnode-sw5-adm-Eth1/44 access link
 switchport mode access
 switchport access vlan 4000
 no shutdown
 !
interface Eth1/41
 description sopnode-sw5-adm-Eth1/41 access link
 switchport mode access
 switchport access vlan 4000
 no shutdown
 !
```

configuration générée

```
graph G {
    "sopnode-sw5-adm" [type="router", vendor="cisco", ip="192.168.1.136", user="admin", password="ansible123"];
    "sopnode-f3" [type="host"];
    "sopnode-f1" [type="host"];

    "sopnode-sw5-adm" -- "sopnode-f3" [int_src="Eth1/44", int_dst="ens15f1"];
    "sopnode-sw5-adm" -- "sopnode-f1" [int_src="Eth1/41", int_dst="ens21f1"];
}
```

topo physique

Architecture modulaire

Code divisé en :

- topology.py
- generator.py
- plugin_loader.py
- main.py

Plugins par constructeur (Cisco, FS, Juniper).

Évolutif : ajout facile de nouveaux constructeurs (un dossier avec un fichier vlan.py).

```
class Cisco:
    def generate(self, access_ports, trunk_ports, allowed_vlans):
        lines = []
        vlan_ids = sorted(set(vlan for _, _, vlan in access_ports))
        for vlan in vlan_ids:
            lines += [f"vlan {vlan}", f" name VLAN{vlan}", "!"]

        for _, iface, vlan in access_ports:
            lines += [
                f"interface {iface}",
                " switchport mode access",
                f" switchport access vlan {vlan}",
                " no shutdown",
                "]"

        vlan_str = ",".join(str(v) for v in allowed_vlans)
        for _, iface in trunk_ports:
            lines += [
                f"interface {iface}",
                " switchport trunk encapsulation dot1q",
                " switchport mode trunk",
                f" switchport trunk allowed vlan {vlan_str}",
                " no shutdown",
                "]"

        return "\n".join(lines)
```

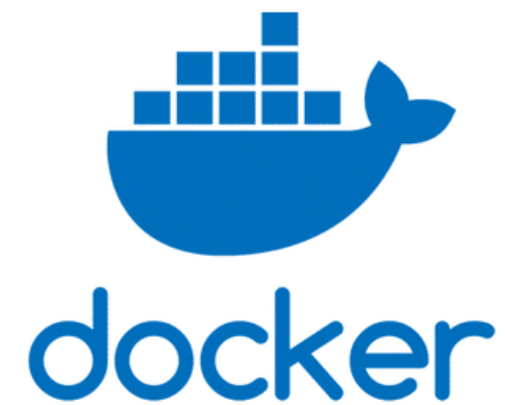
```
└─ ansible
   └─ inventory.ini
   └─ ! push_config.yml
└─ dot-files
   └─ physical.dot
   └─ virtual.dot
└─ plugin
   └─ cisco
   │   └─ vlan.py
   └─ fs
   │   └─ vlan.py
   └─ juniper
   │   └─ vlan.py
└─ templates
   └─ inventory_template.j2
└─ Dockerfile
└─ generator.py
└─ main.py
└─ plugin_loader.py
└─ README.md
└─ requirements.txt
└─ topology.py
```

Intégration avec Ansible, API, Docker

- **Ansible** : push auto vers routeurs (inventaire + playbook).
- **FastAPI** : création d'une API REST pour interagir avec l'outil et l'intégrer à SLICES.
- **Docker** : conteneur complet, exécutable en une commande.



ANSIBLE



Intégration avec Ansible

Ansible : push auto vers routeurs (inventaire *inventory.ini*) + playbook(*push_config.yml*).

```
[cisco]
Cisco1 ansible_host=192.168.1.130
Cisco2 ansible_host=192.168.1.131
Cisco3 ansible_host=192.168.1.132

[cisco:vars]
ansible_network_os=cisco.ios.ios
ansible_user=ansible
ansible_password=ansible123
ansible_connection=network cli
```

inventory.ini

```
- name: Deploy config
  hosts: all
  gather_facts: no

  tasks:
    - name: Push config to router
      ios_config:
        src: "/app/output_configs/{{ inventory_hostname }}_config.txt"

    - name: Save config
      when: save | default(false) | bool
      ios_command:
        commands:
          - write memory
```

push_config.yml

```
(env)-(bohdan@bohdan)-[~/netx/config-deployment/output_configs]
$ cat Cisco1_config.txt
interface FastEthernet1/0
description TEST
!
```

```
(env)-(bohdan@bohdan)-[~/netx/config-deployment]
$ ansible-playbook -i inventory.ini push_config.yml

PLAY [Push Cisco config] *****

TASK [Read config file] *****
ok: [Cisco1]

TASK [Push config to router] *****
[WARNING]: To ensure idempotency and correct diff the input conf
changed: [Cisco1]

PLAY RECAP *****
Cisco1 : ok=2 changed=1 unreachable=0
```

```
interface FastEthernet1/0
description TEST
no ip address
shutdown
duplex auto
speed auto
```

Intégration avec Ansible

Puis j'ai créé un template de fichier inventaire avec Jinja2 pour générer automatiquement le fichier d'inventaire.

Avec l'option de soit utiliser un login et mdp soit une clé pour faire une connexion SSH vers un routeur.

```
# Ansible Inventory - Virtual Topology : {{ virtual_topo_id }}
# Physical Topology : {{ physical_topo_path }}

[routers]
{% for node, data in nodes.items() if data.type == "router" %}
{{ node }} ansible_host={{ data.ip }} ansible_user={{ data.user }}
ansible_connection=network_cli ansible_network_os={{ data.network_os }}{% if
data.password %} ansible_password={{ data.password }}{% endif %}{% if data.key %}
ansible_ssh_private_key_file={{ data.key }}{% endif %}
{% endfor %}
```

```
# Ansible Inventory - Virtual Topology : 038c8a35-939d-4ff6-97be-7c082a166698
# Physical Topology : dot-files/physical.dot

[routers]

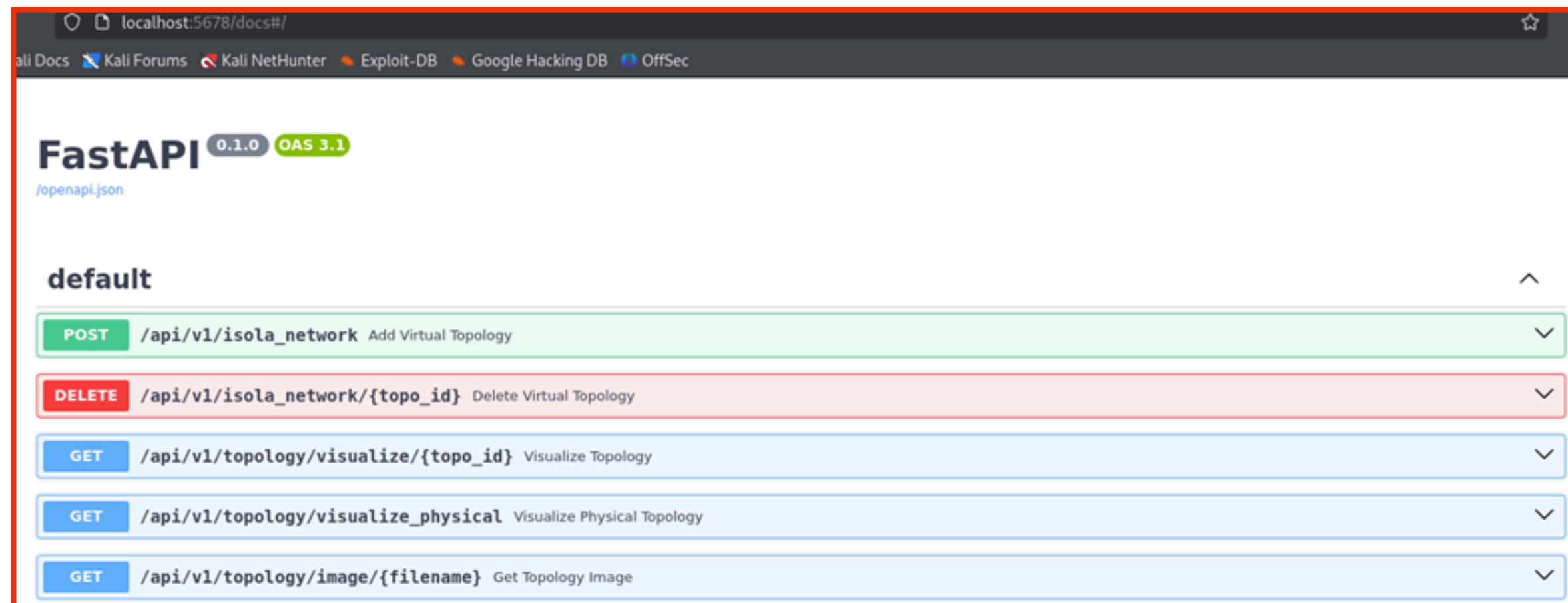
sopnode-sw5-adm ansible_host=192.168.1.136 ansible_user=admin
ansible_connection=network_cli ansible_network_os=cisco.ios.ios
ansible_password=ansible123
```

Intégration avec FastAPI

FastAPI : création d'une API REST pour interagir avec l'outil via l'interface web, une commande curl ou du code.

L'utilisateur peut :

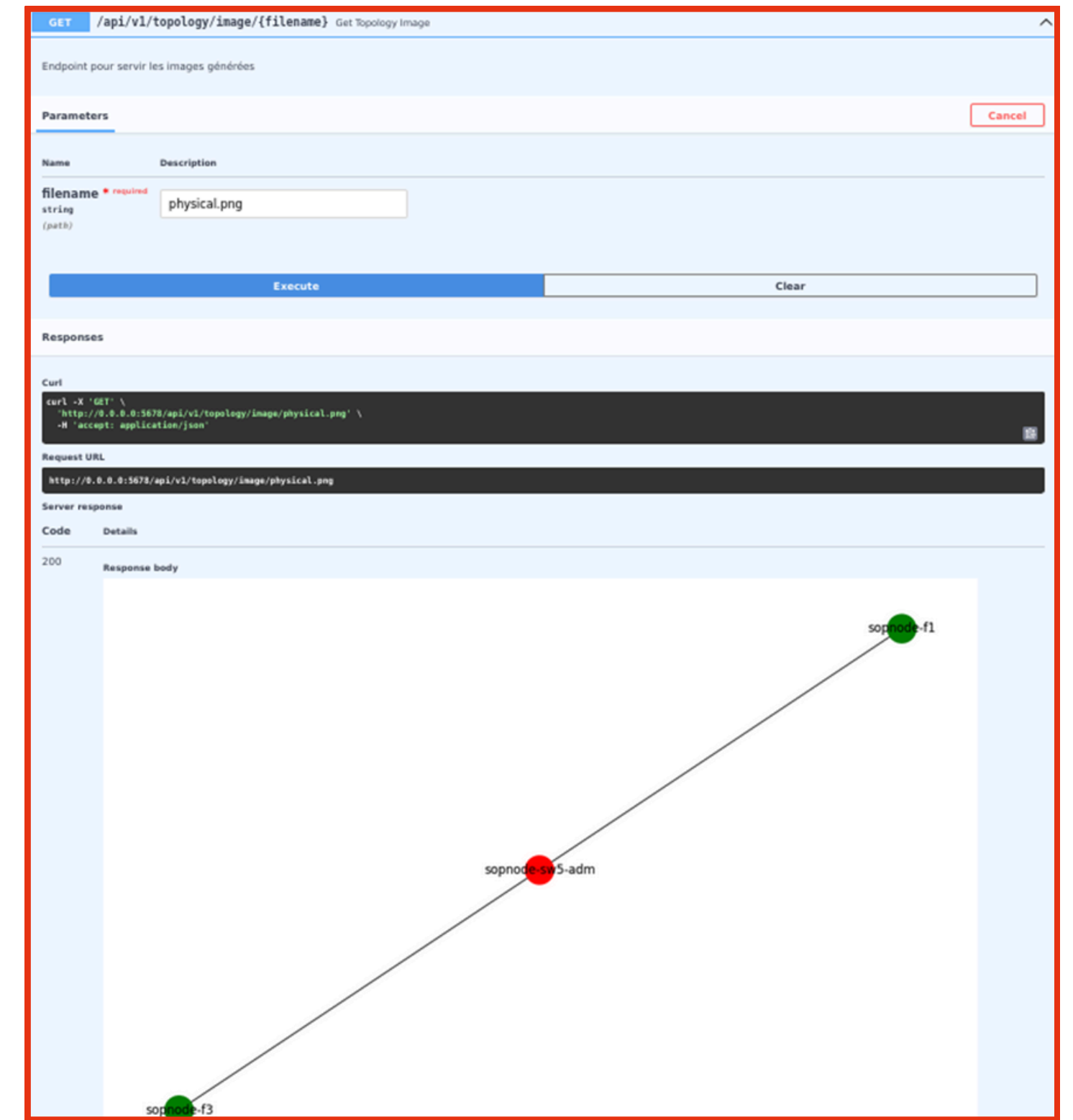
- Via une requête **HTTP POST** envoyer une topologie virtuelle pour que ça génère et déploie les configurations vers routeurs.
- Via une requête **HTTP DELETE** supprimer la topologie virtuelle de la base de données.
- Via une requête **HTTP GET** récupérer et visualiser les topologies physique et virtuel en format *.png*.



Intégration avec FastAPI

Visualisation des topologies

- Visualisation automatique avec matplotlib + networkx.
- Couleurs par type d'équipement (routeur, hôte, unmanaged switch).
- Aide au débogage et à la compréhension des topologies.



Intégration avec Docker

- Création de Dockerfile avec une image de python minimaliste.
- Installation de dépendances.
- `pip freeze > requirements`.
- Montage du répertoire local.
- Lancement de l'API.

```
FROM python:3.12-slim

WORKDIR /app

RUN apt-get update && apt-get install -y \
    gcc \
    graphviz \
    graphviz-dev \
    libgraphviz-dev \
    pkg-config \
    python3-dev \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

COPY . /app
COPY requirements.txt /
RUN pip install --no-cache-dir --upgrade pip
RUN pip install --no-cache-dir -r /requirements.txt

EXPOSE 5678

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "5678", "--reload"]
```

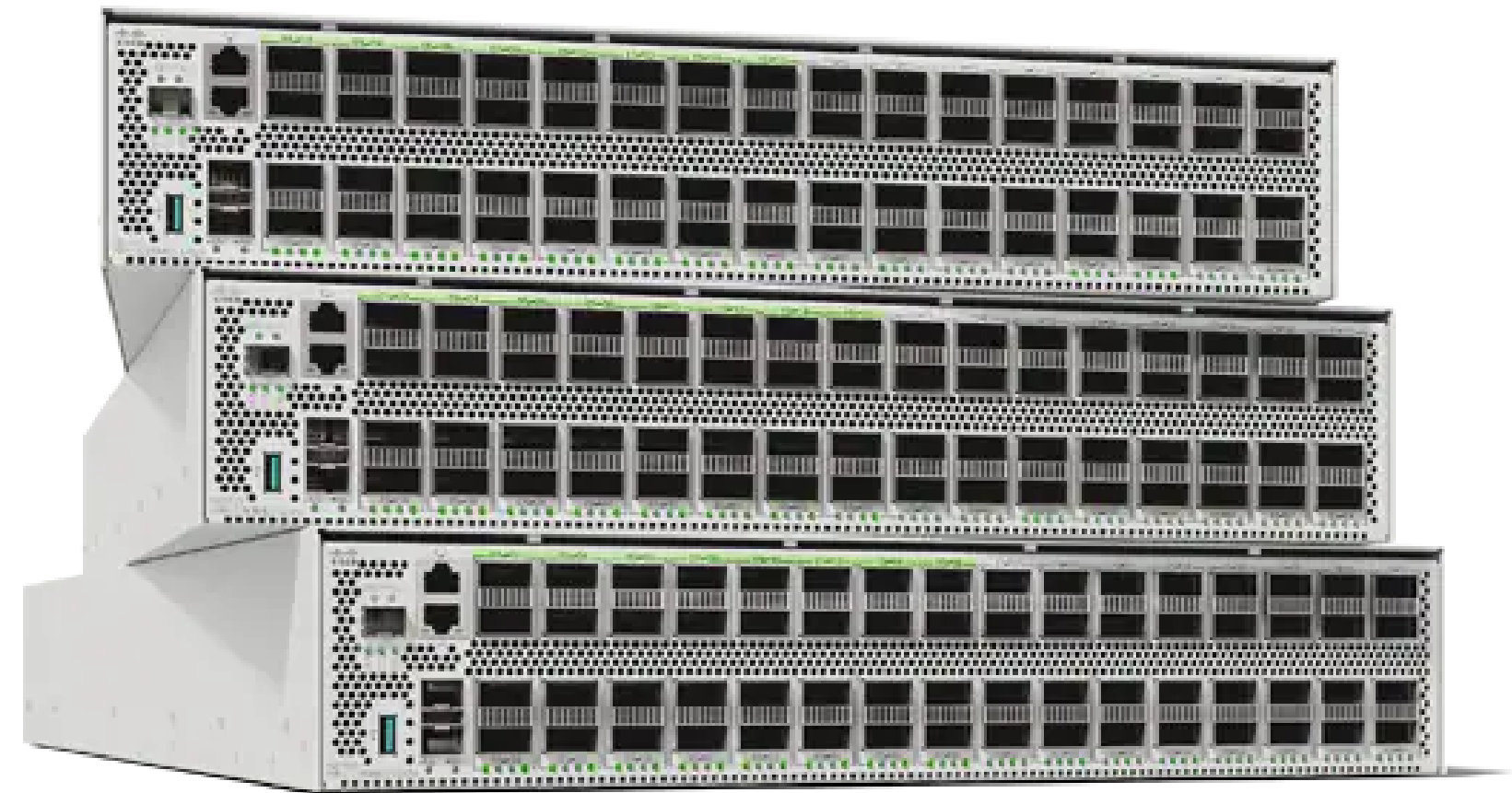
Intégration avec Docker

- Création de Dockerfile avec une image de python minimaliste.
- Installation de dépendances.
- `pip freeze > requirements`.
- Montage du répertoire local.
- Lancement de l'API.

```
(env)-(bohdan@bohdan)-[~/netx/project]
$ sudo docker run -e PORT=5678 -v $(pwd)/data:/data -v $(pwd):/app -p 5678:5678 netx-api
INFO: Will watch for changes in these directories: ['/app']
INFO: Uvicorn running on http://0.0.0.0:5678 (Press CTRL+C to quit)
INFO: Started reloader process [1] using StatReload
INFO: Started server process [8]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 172.17.0.1:57404 - "GET /docs HTTP/1.1" 200 OK
INFO: 172.17.0.1:57404 - "GET /openapi.json HTTP/1.1" 200 OK
> Start compute_and_generate
✓ Base topology : ['virtual_topos/aa2e7995-a586-4ef3-8b1c-87a5de440c93.dot']
✓ Combined virtual topology : 4 nodes
✓ Loaded intermediate model
✓ VLAN map : {'sopnode-f1': 4000, 'sopnode-f3': 4000}
✓ VLANs deducted : {'sopnode-f1': 4000, 'sopnode-f3': 4000}
✓ Loaded plugins : dict_keys(['juniper', 'fs', 'cisco'])
✓ Generated configs for : ['sopnode-sw5-adm']
✓ Wrote files : output_configs/sopnode-sw5-adm_config.txt
✓ Generated inventory.ini : ansible/inventories/inventory_aa2e7995-a586-4ef3-8b1c-87a5de440c93.ini
Deploying Ansible ...
x Error Ansible:
INFO: 172.17.0.1:38074 - "POST /api/v1/isola_network HTTP/1.1" 200 OK
```

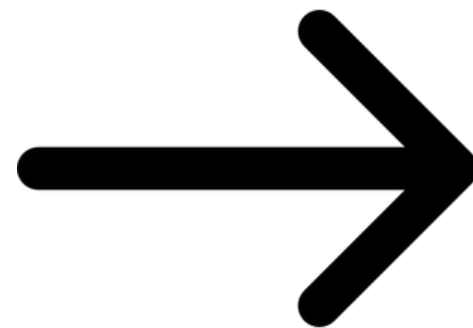
Tests finaux & déploiement

- Environnement simulé : GNS3.
- Environnement réel : tests sur Cisco Nexus 9000.



Résultat final

- Application stable et complète.
- **Entrée** : fichier de topologie virtuelle *.dot*.
- **Sortie** : fichiers de config + déploiement auto.



ANSIBLE

Conclusion et perspectives

- En septembre : prévu d'intégrer l'outil dans l'infra de production.
- Création de modules Ansible pour Juniper / FS.
- Support additionnel de L3 (via VRF).



ANSIBLE
Modules

**Virtual routing
and forwarding**

Remerciements

Mon **maître de stage** - M. Damien Saucez.

Équipe-projet DIANA - Notamment M. Wallid Dabbous et Mme. Christine Foggia.

Mon **tuteur pédagogique** et professeur à l'IUT - M. Guillaume Urvoy-Keller.

Équipe pédagogique et administrative de l'IUT RT.

Merci de votre attention